

Cadence Auto — Manual

Describe a circuit in plain English. Get one that has been simulated.

Repository: https://github.com/Amritha902/cadence_auto

1. What this is

Two tools sharing one circuit engine.

The builder. You type *"build me a half adder"* or *"an op-amp with 60 dB gain and 10 MHz bandwidth"*. It produces a transistor-level netlist, a schematic symbol, a testbench, and **proof that the circuit works** — a truth table for logic, a frequency response for amplifiers. Every number comes out of a real SPICE simulation run at that moment. Nothing is cached, precomputed or asserted.

The benchmark. A reproducible suite for measuring whether language models can size analog circuits better than classical optimizers. This is the research half; it is not part of the web app.

Everything runs on ngspice and open PDKs. No Cadence licence, no NDA, no cloud dependency.

What makes it not a toy

Asking a language model for SPICE produces text that looks right and does not simulate — floating nodes, three-terminal MOSFETs, nets referenced once. Nothing notices until the simulator fails with an error about line 14.

So the model never writes a netlist:

step	who does it
decide what to build	a lookup table, or a model if the table misses
build the transistors	deterministic synthesis in code
check it is a circuit	a structural validator
prove it works	ngspice, against a truth table or a spec

A wrong answer is therefore a wrong *choice*, not a broken circuit — and the last step catches even that.

2. Running it on your laptop

Prerequisites

```
brew install ngspice          # macOS.  Linux: apt install ngspice
```

Setup, once

```
git clone https://github.com/Amritha902/cadence_auto.git
cd cadence_auto
python3 -m venv .venv
.venv/bin/pip install -e '[web,dev]'
```

Get real foundry models (recommended)

```
scripts/fetch_pdk.sh sky130
```

About 20 MB. Without this the app still runs, but only on **dev180** — a hand-written model card that is **not a real process**. Fine for trying things out; never quote its numbers.

Start the web app

```
web/run.sh
```

Open <http://localhost:8010>.

Or use the command line

```
.venv/bin/bias cells                                # what can be built
.venv/bin/bias build "half adder" --out ./out
.venv/bin/bias build "op-amp with 60dB gain, 10MHz bandwidth" --out ./out
.venv/bin/bias sim ota5t --pdk sky130
```

3. Hosting it

The app needs **ngspice**, a system binary. That rules out static hosts like GitHub Pages, Netlify and Vercel's static tier. You need somewhere that runs a container.

Everything needed is already in the repository: [Dockerfile](#), [fly.toml](#), [render.yaml](#).

Option A — Fly.io (recommended)

Scales to zero when nobody is using it, so an unvisited prototype costs nothing.

Step 1 — create an account. Go to <https://fly.io/app/sign-up> and sign up. This is yours to do; it asks for a card for verification even on the free allowance.

Step 2 — install the CLI. Already installed on your machine:

```
flyctl version
```

If it is missing: `brew install flyctl`

Step 3 — sign in. This opens a browser:

```
flyctl auth login
```

Confirm it worked:

```
flyctl auth whoami
```

Step 4 — deploy. From inside the repository:

```
cd ~/cadence_auto
flyctl launch --copy-config --now
```

`--copy-config` tells it to use the committed [fly.toml](#) instead of asking you questions. `--now` deploys immediately.

It will ask you to confirm the app name and region. The config requests **sin** (Singapore); change it if you are elsewhere — `flyctl platform regions` lists them.

Step 5 — open it.

```
flyctl open
flyctl status      # is it healthy?
flyctl logs        # what is it doing?
```

Your URL will be `https://<app-name>.fly.dev`.

Expect the first deploy to take 15–20 minutes. It builds the image from scratch: installing ngspice, fetching the PDK, and verifying a half adder against real foundry models inside the image. Later deploys are much faster because the layers cache.

Option B — Render

Step 1 — sign up at `https://render.com` (GitHub login works).

Step 2 — New -> Blueprint -> pick the `cadence_auto` repository. It reads `render.yaml` and configures itself.

Step 3 — Apply. First build takes a similar 15–20 minutes.

Render's free tier sleeps after inactivity and takes ~30 s to wake. Fly's scale-to-zero is faster.

Option C — any Docker host

```
docker build -t bias-circuits .
docker run -p 8010:8010 bias-circuits
```

563 MB image. Works on Railway, Koyeb, a VPS, or your own machine.

Costs

Fly.io	Free allowance covers a scale-to-zero prototype. Card required at signup.
Render	Free tier works; sleeps when idle.
Your laptop	Free. Only you can reach it.

4. Using it

Type a request and press **Build it**.

Logic — 20 cells

`inverter, buffer, and2, or2, nand2, nand3, nor2, nor3, xor2, xnor2, mux2, decoder2to4, half_adder, full_adder, half_subtractor, full_subtractor, comparator1, majority3, parity4, adder2`

Phrasing is flexible: *"a 2:1 multiplexer"*, *"ripple carry adder"*, *"magnitude comparator"*, *"parity checker"* all resolve.

You get the symbol, the truth table with the **voltage each output actually reached**, and the SPICE netlist. The voltages matter: a gate that settles mid-rail passes a threshold test and is still a broken design.

Amplifiers

Give targets with units:

an op-amp with 60dB gain and 10MHz bandwidth under 100uW driving a 1pF load

Understood units: dB (gain), Hz (bandwidth), W (power), deg (phase margin), F (load).

Three things happen that are worth knowing:

1. ****Constraints you did not ask for are added and labelled `implied`.** Ask only for gain and bandwidth and you still get phase margin $\geq 55^\circ$ and an output that must sit off the rails. Without them the optimizer returns something that meets your request and is not an amplifier.
2. **The topology is chosen for you**, and it tells you why. A single-stage OTA tops out near 48 dB once stability is held; above that it uses a two-stage Miller amplifier.
3. **An impossible target is refused immediately.** Ask for 140 dB and it says so in zero simulations rather than searching and reporting a near miss.

You get the spec table with pass/fail, the sizing, a Bode plot built from 501 simulated points, and the netlist.

5. Troubleshooting

****ngspice: command not found**** `brew install ngspice`, or `apt install ngspice` on Linux.

****bias list shows sky130 as "not fetched"**** Run `scripts/fetch_pdk.sh sky130`. Availability requires every model file to be present, not just some.

****ihp-sg13g2 shows "unusable"**** Correct, and not a bug. Its devices are PSP 103.6 models, which ngspice can only load as a compiled OSDI shared object, and the upstream repository ships none. You would have to build `psp103.osdi` with OpenVAF for your platform. sky130 is BSIM4 and needs no compiled models — use it.

An analog request says "not met" It is telling the truth. The closest result is shown. Either relax a target or raise the budget: `bias build "... " --budget 800`.

The first deploy seems stuck It is not. It installs ngspice, fetches the PDK and verifies a circuit inside the image. 15–20 minutes is normal. `flyctl logs` shows progress.

Deploy fails on the verification step Deliberate. The build runs a half adder against real foundry models and fails if it does not pass, so a broken simulator cannot ship. Read the log — the PDK fetch probably failed.

****git push returns Internal Server Error**** A GitHub-side problem, not yours. Wait and retry; it clears. Quote the Request ID in the error if it persists.

6. What it does not do

Stated plainly, because a tool that oversells itself is worse than none.

- **No layout, no place-and-route.** This is schematic and transistor level. It is not "chip design" in the tapeout sense, and calling it that would be dishonest.
 - **No sequential logic.** No latches or flip-flops yet — they need timing verification rather than a truth table.
 - **Nominal corner only.** One temperature, typical models, no process corners and no Monte Carlo mismatch. A circuit that passes here is not tapeout-ready.
 - **Two analog topologies.** Enough to show a method works; not enough to claim generality.
 - **Twenty logic cells.** Ask for a PLL and it tells you what it has rather than inventing something.
-

7. Where to take it next

In the order I would do them.

1. **Deploy it.** Everything else is secondary — a tool nobody can open has no users. Section 3.
2. **Sequential cells.** D latch, D flip-flop, a small register. The obvious hole for coursework, and they need a different verification approach: setup/hold timing rather than a truth table.
3. **Finish the sky130 spec recalibration.** The benchmark's difficulty tiers are still calibrated against `dev180`. On real models only the easy tiers pass, and it is not yet known which of the rest are physically infeasible versus merely budget-limited. `scripts/calibrate.py 800 3 sky130` answers it; it takes about 40 minutes.
4. **Run the LLM arm.** The benchmark's whole question — can a model beat differential evolution at matched simulation budget? — has never been answered, because it needs an `ANTHROPIC_API_KEY`. `miller-base` and `miller-hard` are unsolved by every classical baseline, which is exactly the headroom to test in.
5. **Process corners.** ss/ff in addition to tt. This changes which sizings are actually good, and is the first step toward numbers a real designer would respect.

A note on the business

This is aimed at **students**, not chip companies — that is a deliberate choice. Selling EDA tools means enterprise procurement, air-gapped networks and 12–18 month sales cycles, and those buyers are not reachable by a link. Students are, and they have the exact problem this solves.

But be clear-eyed: **students do not pay**. This is a distribution and credibility play. Its value is users, a portfolio piece, and a reason for someone in the industry to take a meeting — not revenue.

8. Reference

Commands

```
# local
web/run.sh                                # web app on :8010
.venv/bin/bias cells                      # list logic cells
.venv/bin/bias list                       # topologies, specs, PDKs
.venv/bin/bias build "<request>" --out ./out
.venv/bin/bias sim ota5t --pdk sky130
.venv/bin/bias solve ota5t-base --with de --budget 400
.venv/bin/bias bench --seeds 5
.venv/bin/python -m pytest -q             # 270 tests

# pdks
scripts/fetch_pdk.sh sky130               # ~20MB, the two devices used
scripts/fetch_pdk.sh sky130-full          # ~770MB, everything

# container
docker build -t bias-circuits .
docker run -p 8010:8010 bias-circuits

# fly
flyctl auth login
flyctl launch --copy-config --now
flyctl status / logs / open
flyctl deploy                             # redeploy after changes
```

Layout

```

bias/
  netlist.py    circuit IR: validation, emission, graph identity
  logic.py      deterministic static-CMOS synthesis
  verify.py     truth-table proof by transient simulation
  symbol.py     SVG schematic symbols
  specparse.py  spoken specs -> measurable targets
  design.py     natural language -> verified circuit
  plot.py       AC sweeps and Bode plots
  topology.py   analog circuits and netlist emission
  pdk.py        process abstraction
  sim.py        ngspice driver
  evaluate.py   the single evaluation path, budget counting
  optimizers.py classical sizing baselines
  specs.py      the frozen benchmark suite
  bench.py      the benchmark harness
web/
  server.py     FastAPI routes
  static/       the single-page front end
scripts/
  fetch_pdk.sh  download open PDKs
  calibrate.py  check benchmark specs are reachable
Dockerfile, fly.toml, render.yaml

```

Licence

MIT for the code. PDKs keep their own licences (sky130 is Apache-2.0) and are not redistributed in this repository.